



# **Pitch Unreal Engine Connector™ User's Guide**

Version 1.4

# Table of Contents

|                                                                   |    |
|-------------------------------------------------------------------|----|
| 1. Introduction .....                                             | 2  |
| 1.1. About this document .....                                    | 2  |
| 1.2. About Standard and Free editions .....                       | 2  |
| 2. Concept of Operations .....                                    | 3  |
| 3. Prerequisites .....                                            | 4  |
| 4. Creating an Unreal Engine Project .....                        | 5  |
| 5. Setting up the Unreal Engine Connector Plugin .....            | 6  |
| 5.1. The RPRConnector actor .....                                 | 6  |
| 5.2. Geo-Referencing System .....                                 | 11 |
| 5.3. Connecting to a Federation .....                             | 11 |
| 6. Setting up entity mapping .....                                | 13 |
| 7. Animating 3D models for remote entities .....                  | 14 |
| 7.1. About 3D models .....                                        | 14 |
| 7.2. Preparing the model to handle data from the federation ..... | 15 |
| 7.3. Articulated Parts Animation .....                            | 18 |
| 7.4. Other attributes .....                                       | 19 |
| 7.5. Receiving Interactions .....                                 | 20 |
| 7.6. Human Actor Animations .....                                 | 22 |
| 8. Ground Clamping .....                                          | 23 |
| 9. Publishing .....                                               | 24 |
| 9.1. Publishing Local HLA Entities .....                          | 24 |
| 9.2. Publishing Articulated Parts .....                           | 26 |
| 9.3. Publishing Local HLA Interactions .....                      | 27 |
| 10. Running the project .....                                     | 29 |
| 11. The RPRConnector Sample project .....                         | 30 |
| 11.1. Building the project .....                                  | 30 |
| 11.2. Displaying the Menu UI .....                                | 31 |
| 11.3. User inputs .....                                           | 32 |
| 11.4. Displaying the Entity Identification Widget .....           | 32 |
| 11.5. Displaying the Entity Information Widget .....              | 33 |
| 11.6. Point of view .....                                         | 33 |
| 11.7. Working with previous versions of Unreal Engine .....       | 34 |
| 12. Troubleshooting & known issues .....                          | 35 |
| 12.1. Set Actor Location and Rotation in blueprint model .....    | 35 |

# 1. Introduction

## 1.1. About this document

The Pitch Unreal Engine Connector™ Plugin, Standard or Free Edition, allows an Unreal Engine game to connect to a HLA/RPR2 FOM Federation to receive entity updates and interactions, as well as publish Unreal Engine actors to the HLA Federation. The plugin handles the spawning of actors, chosen via the entity mapping file, as well as updating their HLA specific attributes when updates are received from the federation. It also handles the publishing of HLA specific attributes from Unreal Engine actors.

This document outlines a step-by-step guide to using the Pitch Unreal Engine Connector™ Plugin for the first time. Following these steps will create a basic Unreal Engine project that connects to a HLA Federation, spawns remote HLA entities as Unreal actors, and notifies these actors when there is an update from the HLA Federation.

## 1.2. About Standard and Free editions

Pitch Unreal Engine Connector™ is available in two editions: the commercial Standard Edition and the Free Edition.

The **Standard Edition** package includes:

- The plugin and its source code
- A sample UE C++ project, see [Section 11](#) section
- User and Developer documentation

The **Standard Edition** requires a license key. Refer to [Licensing](#) section in Developers guide for more information.

The **Free Edition** package includes:

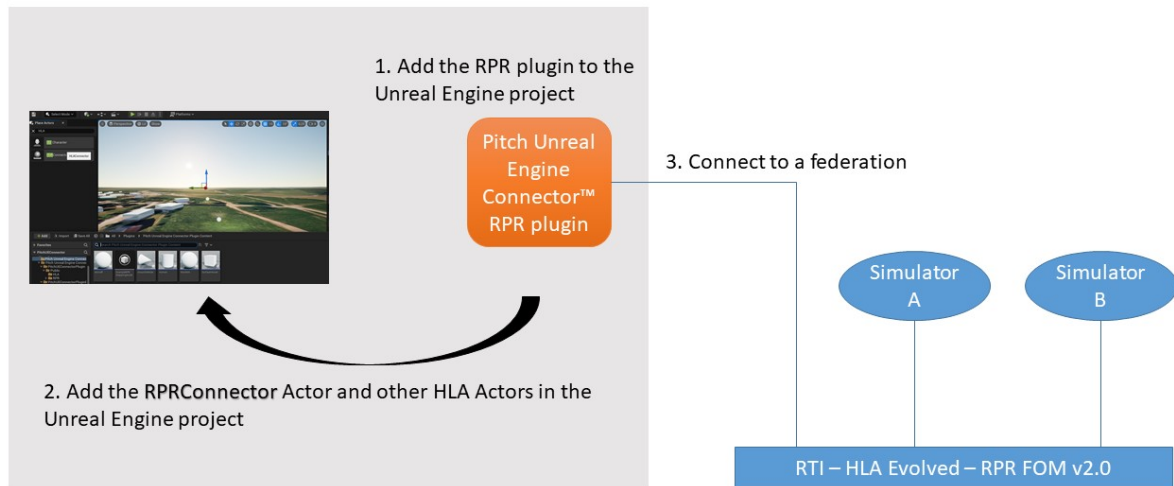
- The plugin, only binaries, built using *Development Editor* target.
- User documentation

The **Free Edition** has some limitations:

- DIS is not supported
- Publishing objects and interactions from Unreal game is not supported

## 2. Concept of Operations

The Pitch Unreal Engine Connector™ Plugin allows an Unreal Engine game to connect to a HLA Federation to receive entity updates and interactions, as well as publish Unreal Engine actors to the HLA Federation.



*Figure 1. Using Pitch Unreal Engine Connector™ RPR plugin in a Unreal Engine project.*

1. Add the Pitch Unreal Engine Connector RPR plugin to the project, see [Section 4](#).
2. Add the RPR Connector Actor and other actors in the Unreal Engine project, see [Section 5](#).
3. And finally, connect the Unreal Engine game to the federation, see [Section 5.3](#). Then objects and interactions published by other federates such as CGF or other simulators can be displayed in the game, see [Section 7](#), and Unreal actors can be published to the federation, see [Section 9](#).

A HLA Evolved Run-Time infrastructure, such as Pitch pRTI™, is required to connect to a HLA federation.

### 3. Prerequisites

- Unreal Engine (Version 5.1 or above)
- HLA Evolved compliant Run-Time Infrastructure (RTI)
- Visual Studio (2019 or above), refer to Unreal Engine documentation to configure the build chain

The Pitch Unreal Engine Connector™ RPR plugin requires the GeoReferencing plugin. Enable it from the Plugin browser tab in the Unreal Engine editor.

#### NOTE

##### *Working with different versions of Unreal Engine*

The packages provided has been developed using Unreal Engine 5.5.

However, with the **Standard Edition**, it is possible to work with a previous Unreal Engine version, see [Section 11.7](#)

Using a newer version of Unreal Engine may require some changes in the code, refer to Unreal Engine documentation.

The **Free Edition** works only with UE 5.5, in *Development Editor* mode.

## 4. Creating an Unreal Engine Project

Firstly, an Unreal Engine project need to be created, and the Pitch Unreal Engine Connector™ Plugin added to the project.

### NOTE

Standard Edition users can skip this step and use the sample PitchUEConnector.uproject provided, see [Section 11](#) section.

1. Creating a new Unreal Engine Project
  - a. Open the Unreal Engine Editor
  - b. Select the *Games* or *Simulation* tab on the left
  - c. Choose the *Blank* or *Simulation Blank* template
  - d. In the Project Defaults, select to use a C++ project, and give your project a name (In this document, this new project is named "**PitchDemo**")
  - e. Click *Create*
  - f. This will have created a new Unreal Engine project, it will also open the Unreal Editor as well as Visual Studio. For now, close them both.
2. Add the Unreal Engine Connector Plugin to the project
  - a. In the project directory (E.g. <Path to Unreal Projects folder>/PitchDemo), create a new folder called Plugins, if one does not exist already
  - b. Copy the Unreal Engine Connector Plugin (E.g. the folder containing the PitchUEConnectorPlugin.uplugin file) into the new Plugins folder of your project
  - c. Right click PitchDemo.uproject and select *Generated Visual Studio project files*
  - d. Double click PitchDemo.sln to open the Visual Studio solution.
  - e. In PitchDemo.Target.cs and PitchDemoEditor.Target.cs add these lines

```
// Force ANSI allocator for game client
GlobalDefinitions.Add("FORCE_ANSI_ALLOCATOR=1");
bOverrideBuildEnvironment = true;
// Force enable exceptions for c++ exception handling
bForceEnableExceptions = true;
```

- f. For Standard Edition users, add the license key as described in *Licensing* section in the Developers guide.
- g. In Visual Studio, go *Build* → *Build Solution*. This will build the Unreal Engine project and the Unreal Engine Connector Plugin.

## 5. Setting up the Unreal Engine Connector Plugin

After opening the project in the Unreal Engine Editor, a level has to be created. In this level, it is possible to add a map representing the terrain or use a terrain streamer plugin.

### 5.1. The RPRConnector actor

To set up the Pitch Unreal Engine Connector™ plugin, the first step is to add the **RPRConnector** actor in the level and configure it.

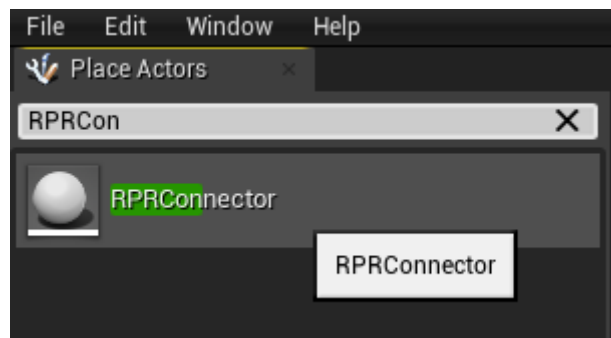


Figure 2. How to get the RPRConnector actor.

To configure the RPRConnector actor, select it in the Outliner panel.

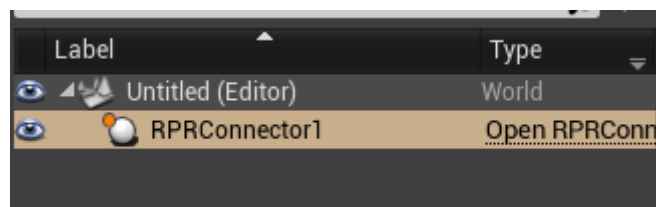


Figure 3. The RPRConnector actor in the Outliner.

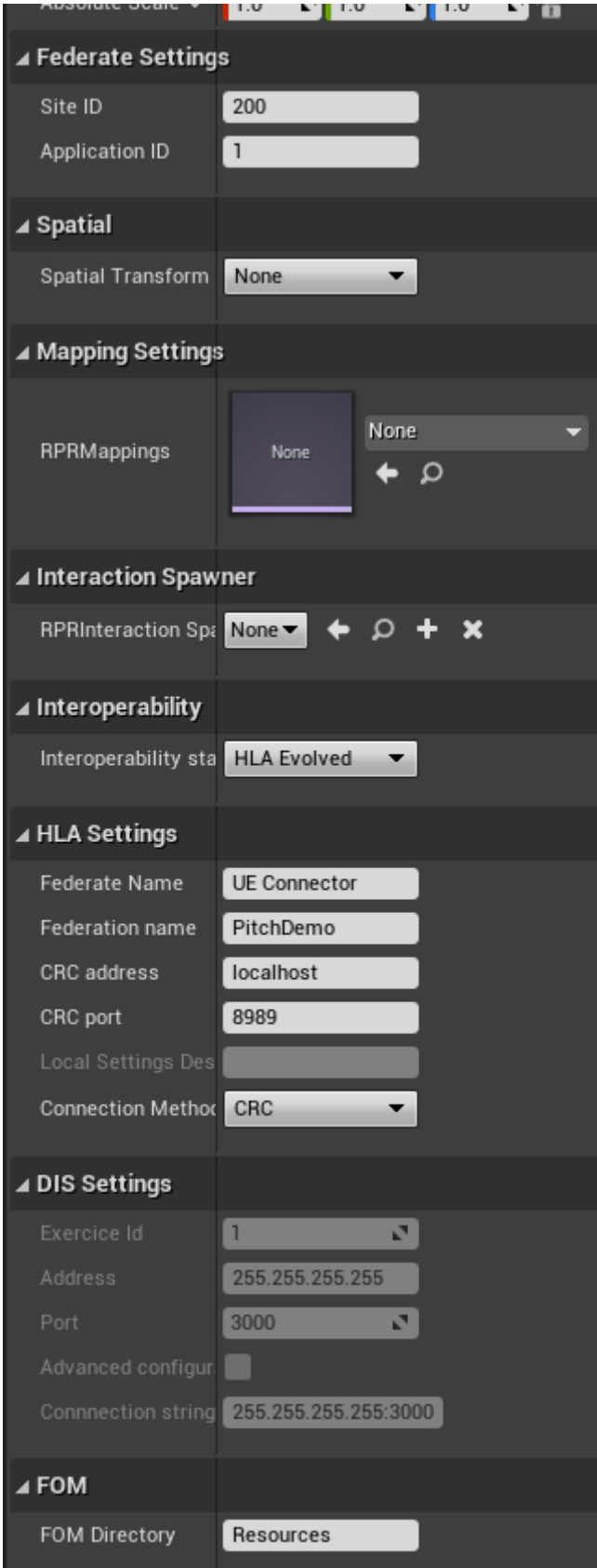


Figure 4. The RPRConnector settings.

In the Details panel, the main items to configure are:



- The Interoperability standard to use: HLA Evolved or DIS.
- For the HLA Evolved standard, the HLA Settings with:

| Settings                  | Description                                                   |
|---------------------------|---------------------------------------------------------------|
| Federate Name             | The name of the federate                                      |
| Federation Name           | The name of the federation to join                            |
| Connection Method         | The connection method: select <b>CRC</b> or <b>LSD</b>        |
| CRC Address               | The address of the CRC                                        |
| CRC Port                  | The port of the CRC                                           |
| Local Settings Designator | The RTI connection string to use as Local Settings Designator |

The **CRC** connection method is specific to connect easily to Pitch pRTI™.

The **LSD** connection method can be used for any RTI. Please refer to the RTI documentation.

- For the DIS standard, the DIS settings are:

| Settings               | Description                                                          |
|------------------------|----------------------------------------------------------------------|
| Exercise Id            | The id of the DIS exercise                                           |
| Address                | The network address of the DIS exercise                              |
| Port                   | The send and receive port number of the DIS exercise                 |
| Advanced configuration | Check if other connection settings are needed                        |
| Connection string      | Use this connection string to connect to the DIS exercise, see below |

- DIS Advanced configuration

In Advanced configuration mode, the *Connection string* parameter shall be formatted as:

```
address[:send port][,interface[:receivePort]][;additional configuration flags]
```

Several configuration flags can be appended to the *Connection string* parameter. Semicolon should be used to separate the settings string and the individual flags. The following flags are supported:

| Flag                   | Description                                                                                                      |
|------------------------|------------------------------------------------------------------------------------------------------------------|
| HEARTBEAT=<number>     | Heartbeat interval in seconds.<br>Heartbeats are disabled if set to 0.<br>Default is every 5 s                   |
| DELETE=<number>        | Timeout for deleted instances.<br>Default is after 13 s.                                                         |
| DIS_VERSION=<number>   | DIS version.<br>Default is version 6                                                                             |
| LOOPBACK               | Allow loopback messages from the same application. This includes all the sent messages.<br><br>Default is false. |
| DEBUG                  | Print debug messages on the console. This should be enabled during development and testing.<br>Default is false. |
| MULTICAST_TTL=<number> | TTL (time to live) for multicast UDP packets.<br>Default is defined by the operating system, usually 1.          |

The *Connection string* with DIS version set to 7, loopback and debug enabled could look like this:

```
192.168.255.255:3030;LOOPBACK;DEBUG;DIS_VERSION=7
```

- Federate Settings

| Settings       | Description                                                              |
|----------------|--------------------------------------------------------------------------|
| Site Id        | The identifier of the site, see IEEE 1278.1-1995 section 5.2.14.1        |
| Application Id | The identifier of the application, see IEEE 1278.1-1995 section 5.2.14.1 |

- Spatial Settings

| Settings                         | Description                                                                                                                                                   |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Spatial Transformation Algorithm | Whether to apply any spatial transformation on the spatial data received. The options are to apply none, apply dead reckoning or apply a smoothing algorithm. |

| Settings                            | Description                                                                 |
|-------------------------------------|-----------------------------------------------------------------------------|
| Ground Clamping Trace Line Distance | Distance between ray tracing starting point and entity location, in meters. |

- Mapping Settings

Select an asset which is based on the `RPRMappingAsset` class provided by the plugin. The sample provides an example of such asset: **ExampleRPRMappingAsset**.

This asset is needed to associate a RPR entity type, identified using a septuplet (see IEEE 1278.1-1995 section 5.2.16) and a 3D model in the Unreal Engine World.

Any asset which implements the **RPREntity Interface** could be used to map an RPR entity type, see [\[Using 3D models\]](#).

For example, to set this in a blueprint actor, open the asset in the editor, open Class Settings then, in Details panel, add The **RPREntity Interface** to the *Interfaces* section.

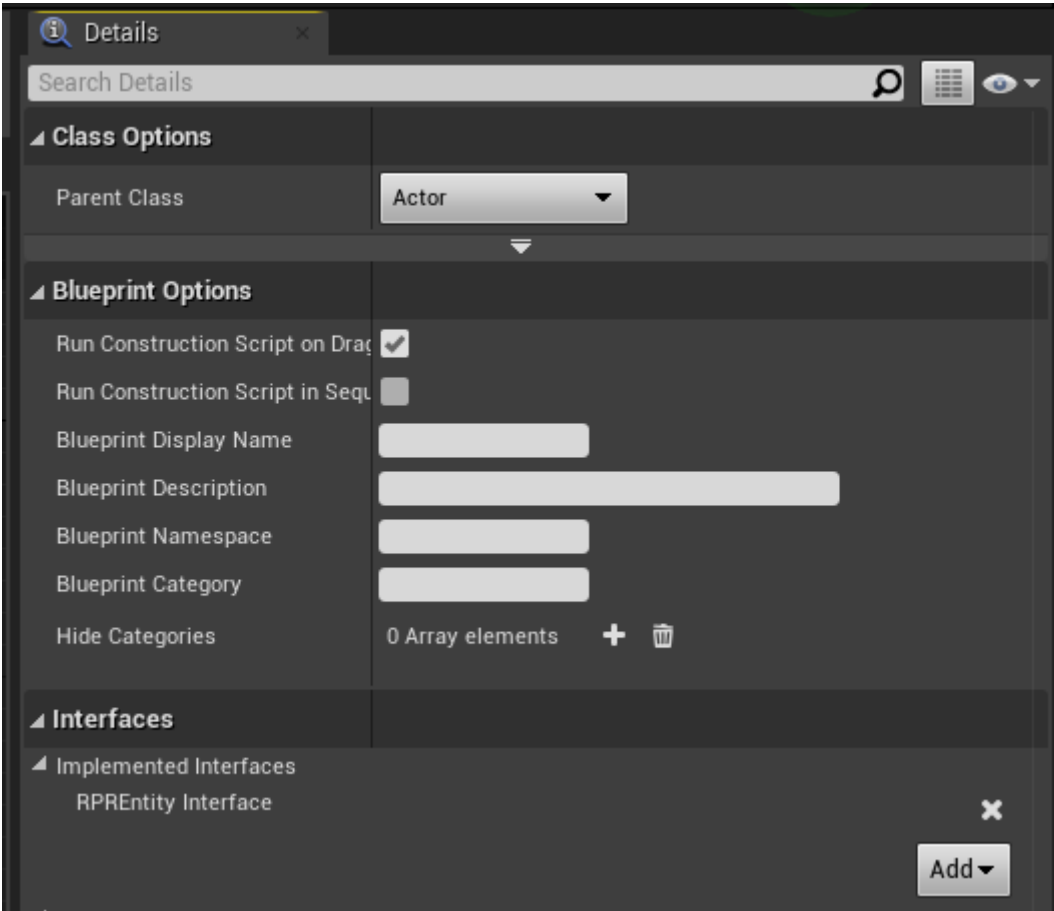


Figure 5. Adding **RPREntity Interface** to the actor.

- Manager Classes

Select the class that handles the spawning, updating, destroying of actors. HLA listeners

are also set up here.

This class is FOM Specific and only a RPR FOM example is provided in the plugin (**RPRObjectClassManager**). To create other class managers for different FOMs, create a new class that inherits from **HLAObjectClassesManager**.

- Interaction Spawner

The actor that will spawn any interaction particles (E.g. Explosions). An example is provided with the sample (**InteractionSpawner**).

## 5.2. Geo-Referencing System

It is necessary to add a **Geo Referencing System** actor in the level.

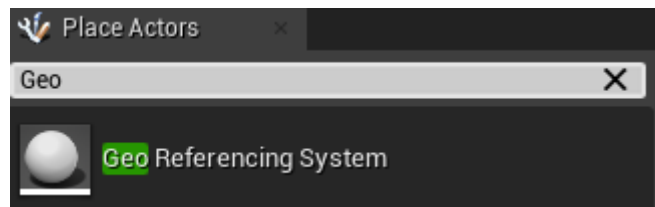


Figure 6. Place a **Geo Referencing System** actor in the level.

See Unreal Engine documentation for more details.

## 5.3. Connecting to a Federation

To connect to a federation the **JoinFederation** function must be called. This can be done at any point during runtime. For example, it could be done on **BeginPlay** in the level blueprint, which would connect as soon as the project is run, or after some menu input if there is a UI.

There are two options when calling this function:

| Function                              | Description                                                                                                      |
|---------------------------------------|------------------------------------------------------------------------------------------------------------------|
| Join Federation                       | This function takes a number of connection parameters allowing for a custom federation to be joined at any time. |
| Join Federation Using Property Values | This function takes no connection parameters and will use the property values of the RPR Connector               |

The image below shows how to access both of these function in blueprints

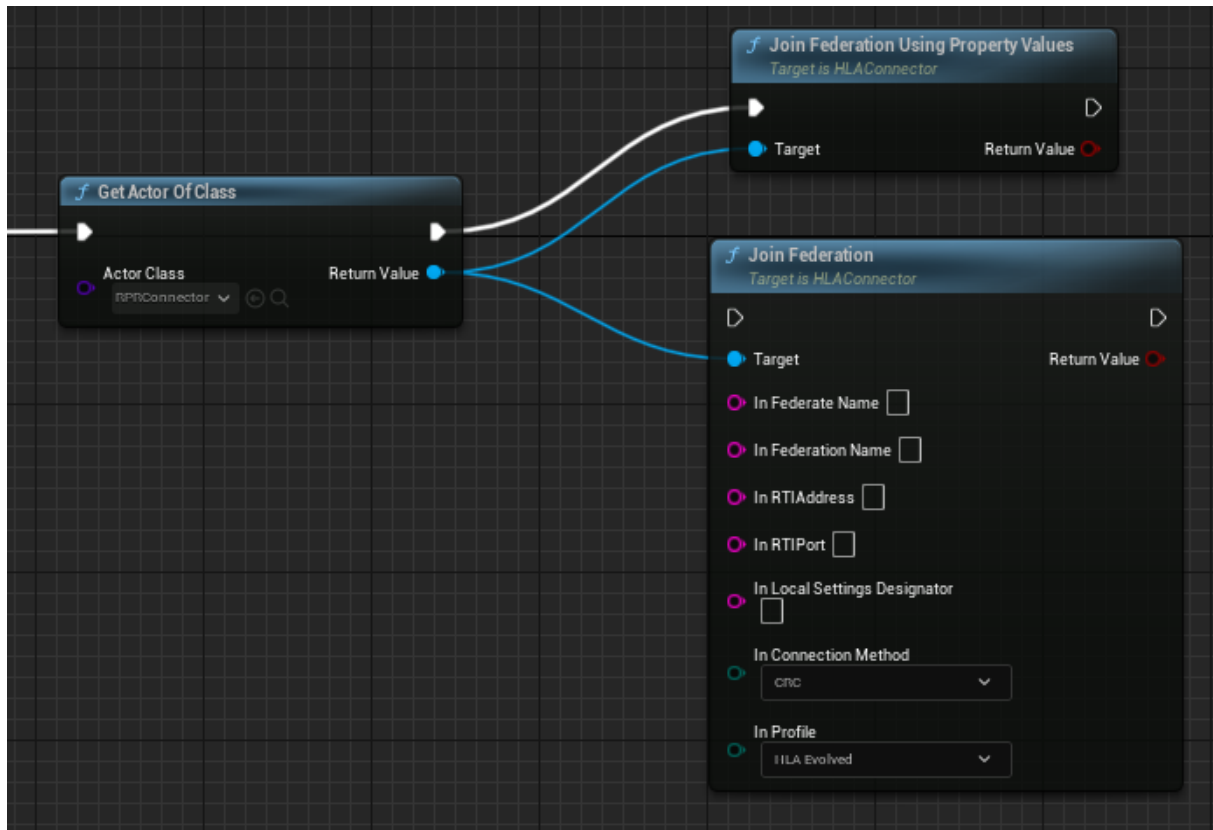


Figure 7. How to join federation.

## 6. Setting up entity mapping

Before runtime, a **RPRMapping** asset needs to be created in the editor. This allows for default entity mapping to be set. For each entity type set, the blueprint to be spawned for that entity type can be chosen.

These blueprints must implement the **RPREntityInterface**, see [Section 7](#), marking them as compatible with RPR and HLA.



Figure 8. Define entity mapping with the RPRMapping asset.

When the game begins, the default mapping from the chosen RPR Mapping file will be read and stored for use when spawning new assets.

When an actor is spawned, the blueprint spawned is chosen by checking for any close matches in the default mappings. For example, if the default mapping shown above is set, then any entity with an entity type that begins with 1.2 (E.g. 1.2.3.4.5.6.7 or 1.2.0.0.1.2.1) will spawn GameActorBP.

If there are no matches between the default mapping and an entity's entity type, then the actor will not be spawned. This is because the plugin does not know which RPR actor component the actor should be using. This could cause a mismatch between which attributes the RPR actor component is expecting and the attributes being received from the federation.

An example of **RPRMapping** asset, called ExampleRPRMappingAsset is provided in the **Content** folder of the plugin.

## 7. Animating 3D models for remote entities

### 7.1. About 3D models

3D models are provided by different vendors, for free or not.

There is no standard way to set properties for the models, it is up to each model's developer to set/name the interfaces or the blueprint methods to animate the model.

The figures below shows different ways to set turret angles values in model blueprint.

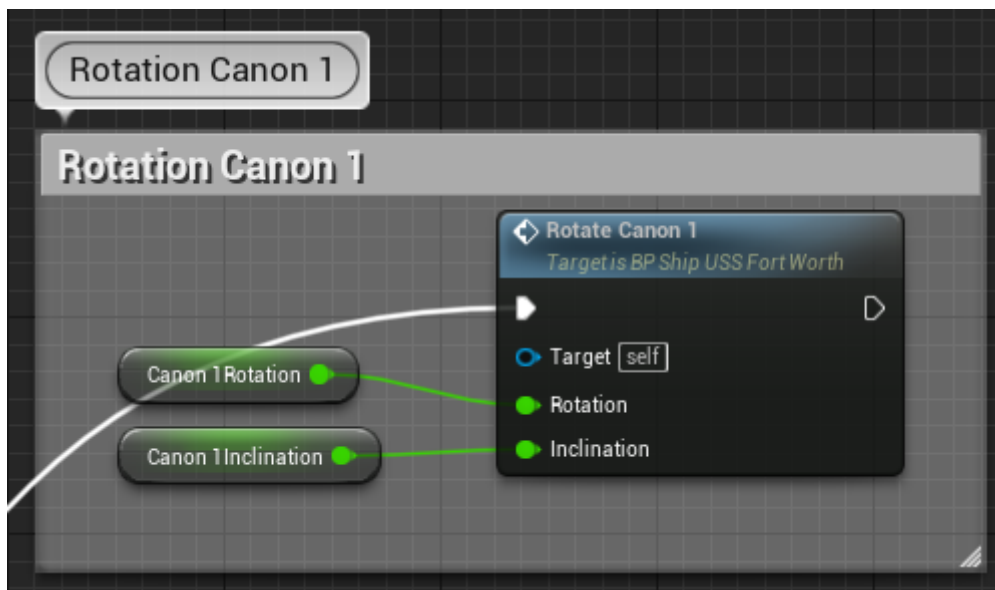


Figure 9. Controlling articulated part angles (azimuth, elevation) using one blueprint action.

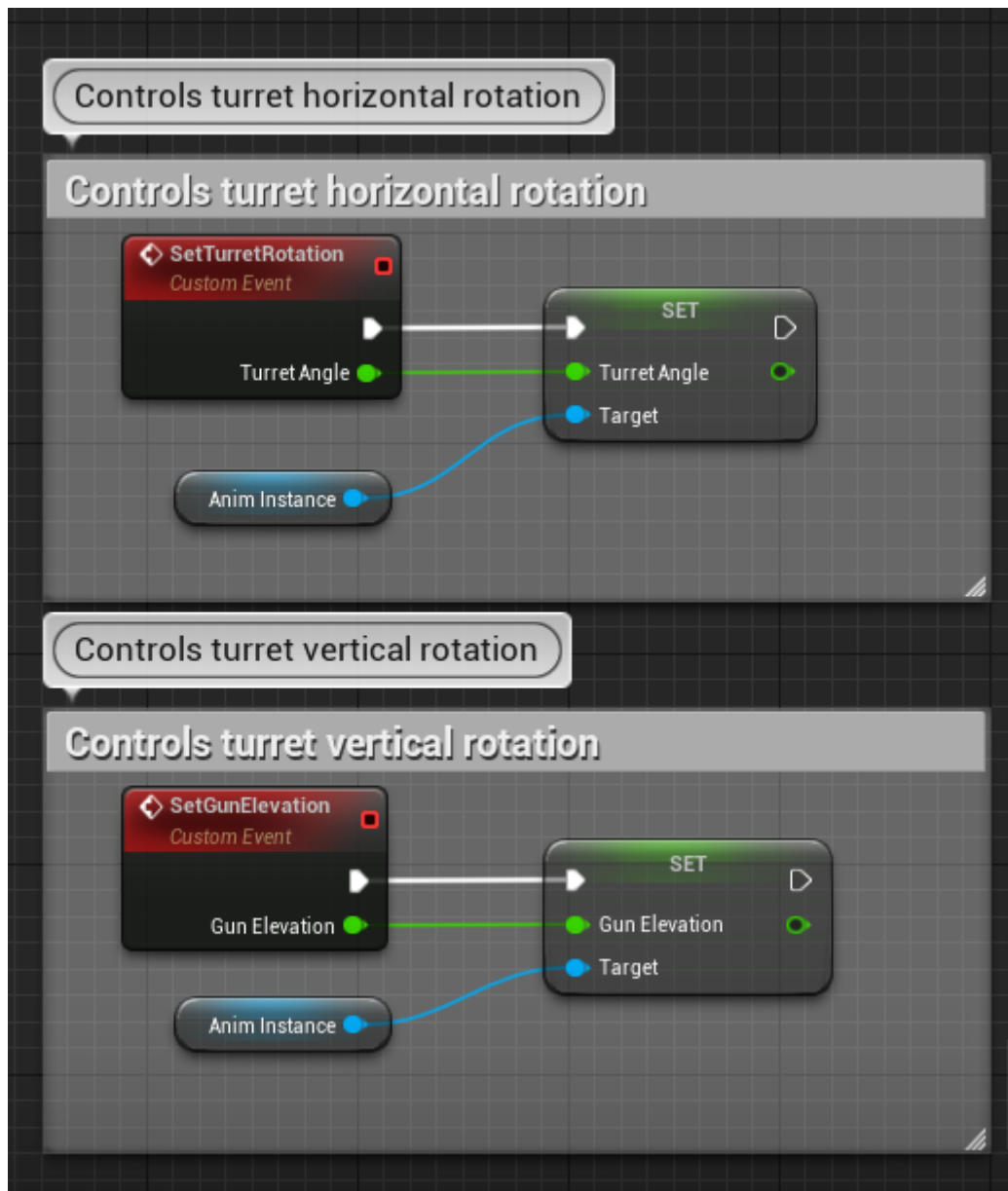


Figure 10. Controlling turret angles separately using two blueprint actions.

## 7.2. Preparing the model to handle data from the federation

To be able use data received from the federation, it is necessary to prepare the model. This operation has 2 steps:

1. The model shall implement the **RPREntity Interface**:

Open the model blueprint in the editor, open **Class Settings** then, in Details panel, add The **RPREntity Interface** to the **Interfaces\Implemented Interfaces** section.



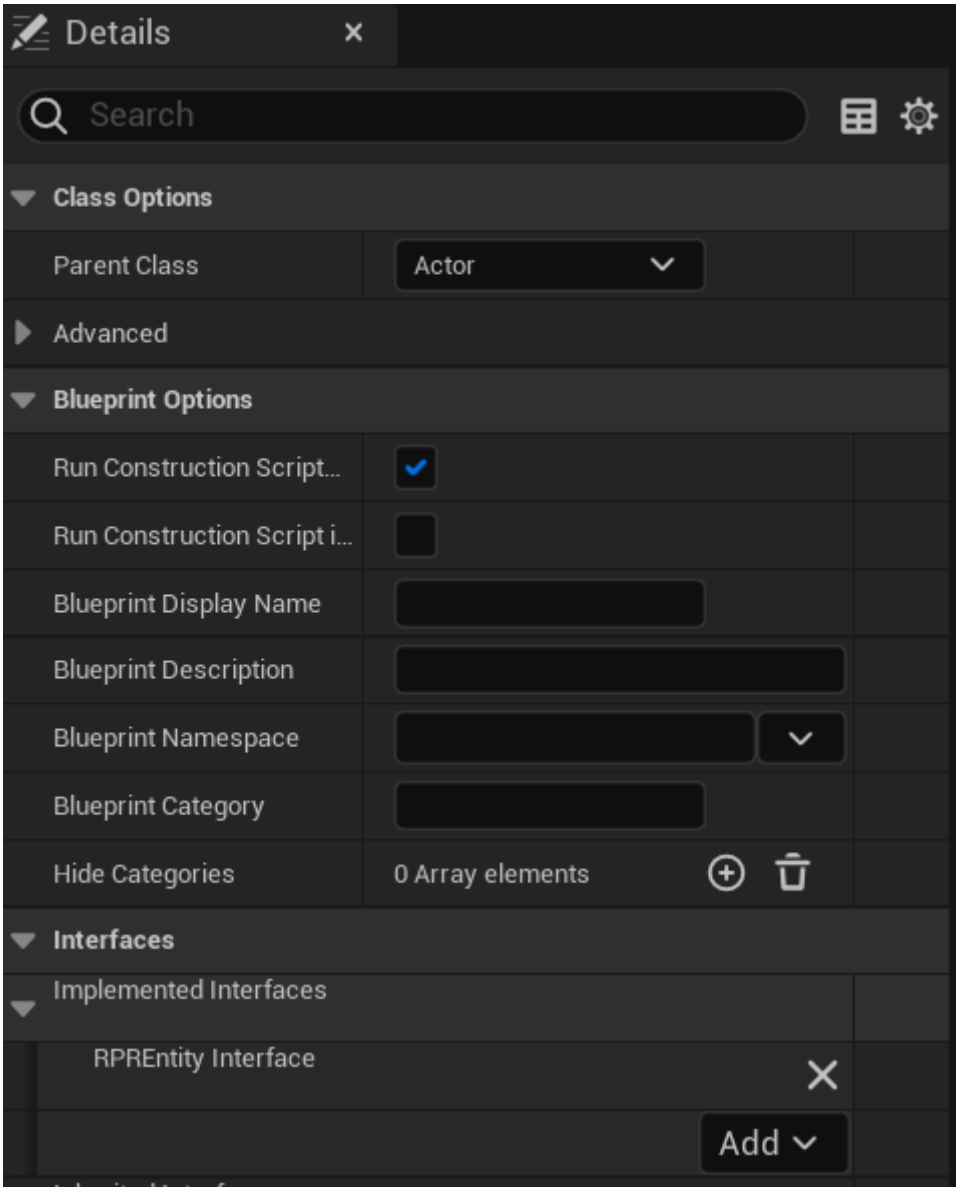


Figure 11. Adding RPREntity Interface to the model.

2. The model shall have a **RPRComponent**:

In the model blueprint editor, in the Component tab, click on the +Add button and select the **RPRComponent** that fits with the model (type RPR in the Search Components bar to filter). The components provided by the plugin reflect the class hierarchy of the RPR-FOM.

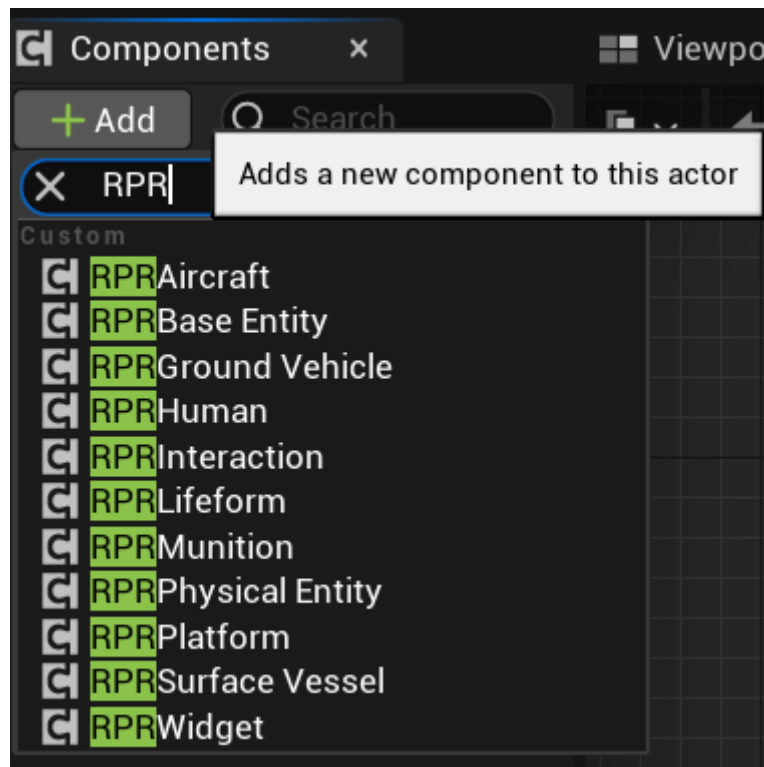


Figure 12. Adding a **RPRComponent** to the model.

Once this preparation done, it is possible to use RPR properties and events provided by the plugin.

Also, this component allows to select some options for the computation of the location of the actor.

The options for computation of location and rotation are:

| Properties                     | Description                                                                                                                                                                                                            |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Internal Location Rotation | If true, the plugin computes and set the location and the rotation in the Unreal World of the actor. Otherwise, the plugin provides only the Geocentric position and its orientation.                                  |
| Use Ground Clamping            | If true, the plugin calculates a new spatial for the entity so that it is on the floor of the terrain. As long as the terrain has block GroundClamp enabled in its collision settings, see <a href="#">Section 8</a> . |

*Using Drivable vehicles*

Some additional preparation may be needed when mapping a drivable model, using for example, Chaos Vehicles plugin. These vehicles expect to use inputs to compute their movement.

Make sure when using such models to map remote entities to disable these feature, for example, on event `BeginPlay`:

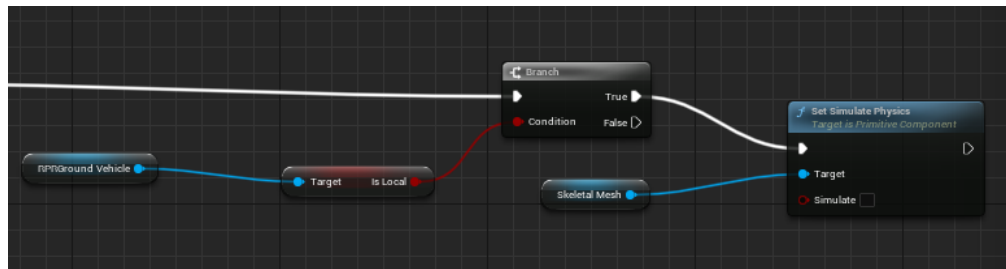
**NOTE**

Figure 13. Disable Simulate Physics for remote entity

Also when setting remote entity location and rotation, it may be recommended to set the Teleport flag to true.

## 7.3. Articulated Parts Animation

Articulated parts data can be used to animate blueprints that support movement of their smaller parts. The articulated parts data is updated using the **OnRPRPhysicalEntityArticulatedPartUpdated** delegate.

This event is handled in the model blueprint, like in the figure below:

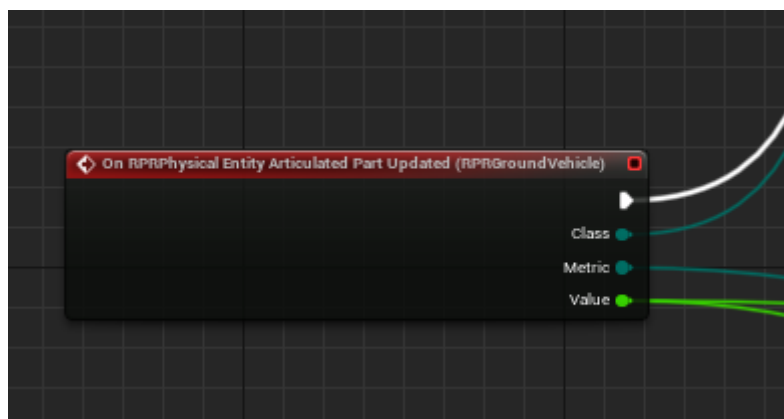


Figure 14. Handling `OnRPRPhysicalEntityArticulatedPartUpdated` event.

Link this event to the **Switch on ArticulatedPartsTypeEnum** and then **Switch on ArticulatedTypeMetricEnum** with the relevant classes and metrics. And finally link to the actions or variables provided by the model to set the different values of the articulated part.

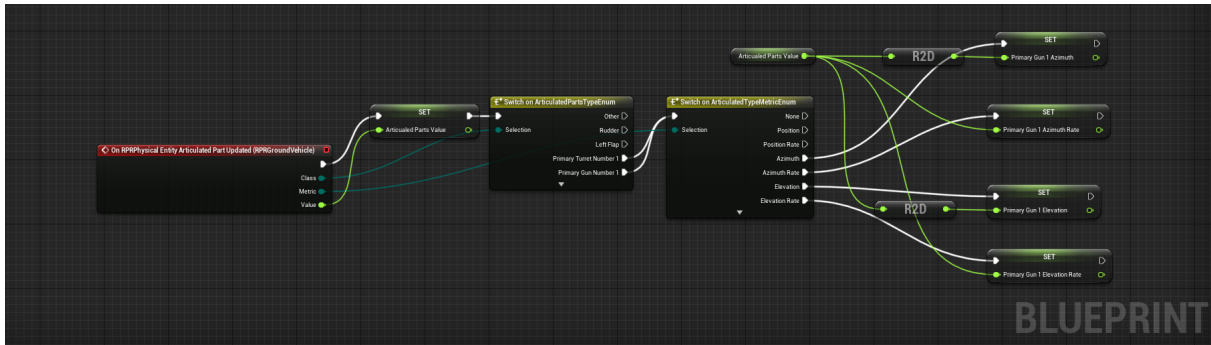


Figure 15. Set articulated part values.

In the example above, the articulated parts data is received through the delegate. The Class and Metric of the updated part is found and local variables are updated based on the received new value for the part. Blueprint function can then be used to update the moving parts of the blueprint actor. As shown below:

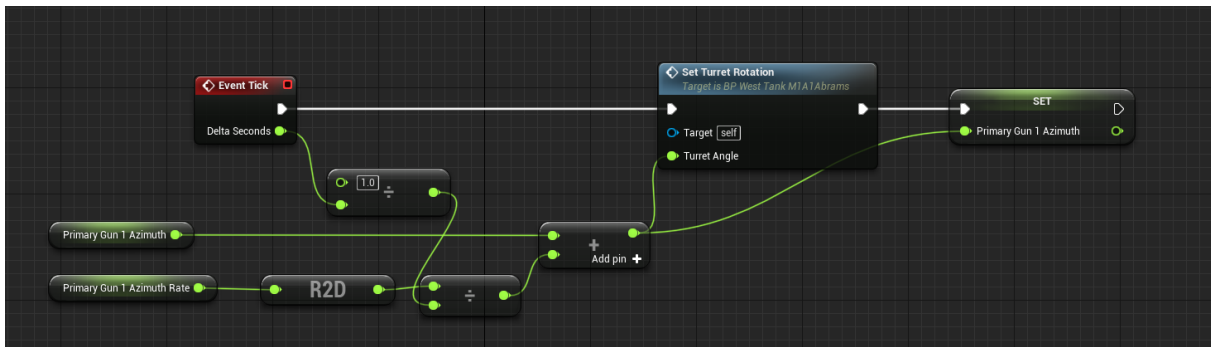


Figure 16. Use local variables to compute articulated part values on model tick event.

## 7.4. Other attributes

It is possible also to get values of the attributes without handling an event, using getters. These attributes can be used to change the actor appearance or to animate some elements of the model such as lights for example. To do this:

1. In the model blueprint, get the **RPRComponent**, using right-click in the blueprint and search for GetRPR.

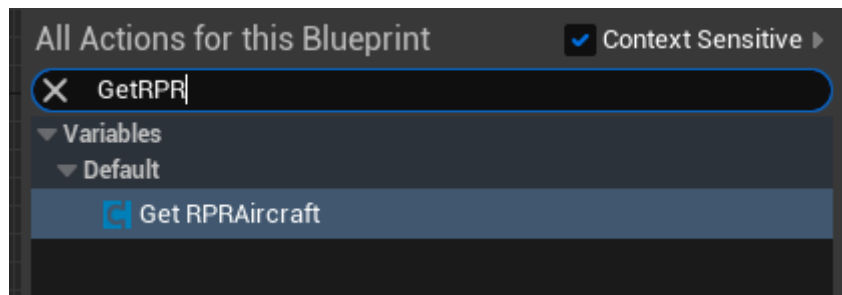


Figure 17. Get the RPRComponent, in that case as a RPR Aircraft.

2. Then from the **RPRComponent** node, draw a new link and search for the desired RPR

variable.

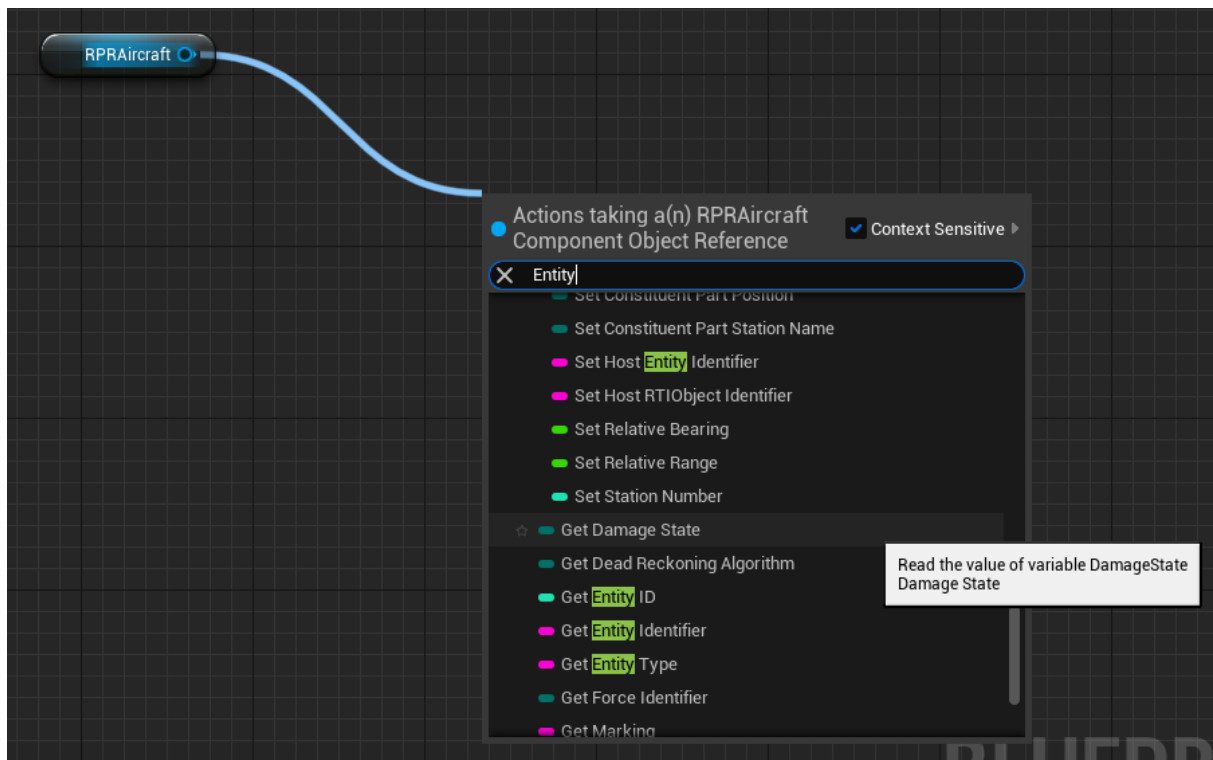


Figure 18. Get an variable for the **RPRComponent**, for example the **DamageState** attribute value.

3. Then, use the selected variable to animate the model

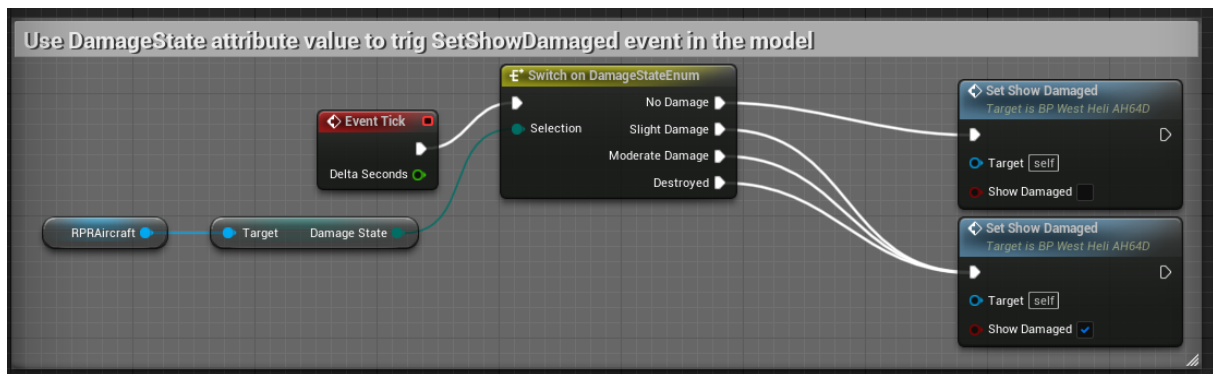


Figure 19. Animate the model, here, for example, trig the **SetShowDamaged** event.

The attributes are in categories **Entity** attributes or **Entity appearance** attributes.

## 7.5. Receiving Interactions

### 7.5.1. Supported interactions

The supported interactions in this version are:

- WeaponFire
- MunitionDetonation

- StartResume
- StopFreeze

### 7.5.2. Handling interactions in blueprints

In the interaction manager there are delegates for the supported interactions. When an interaction is received the delegate will be fired, the struct with the interaction data inside needs to be taken from the RPR Interaction Component. This is due to only nine parameters being allowed with delegates. As the interactions have more parameters than this, another function is needed to get the data.<sup>[1]</sup>

The interaction delegates shall be bounded to a custom event in the blueprint.

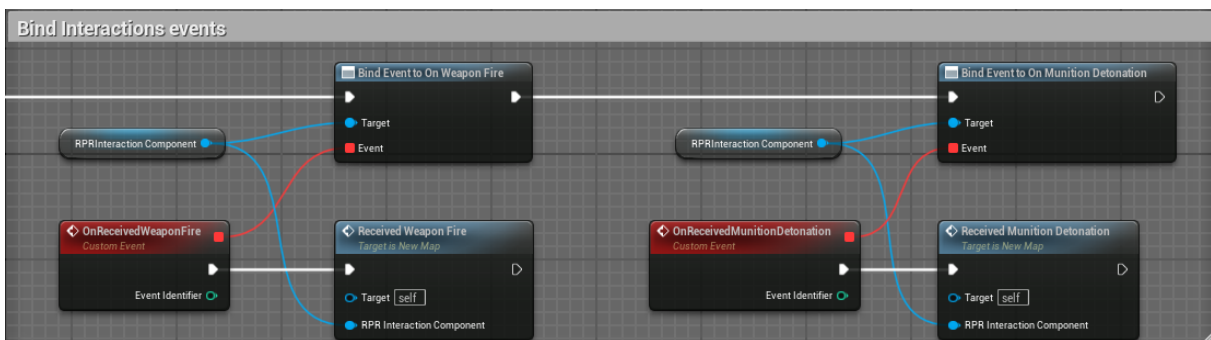


Figure 20. Binding delegates to blueprint custom events.

Then in the bounded function, the data is retrieved calling **GetLastReceivedInteraction** function.

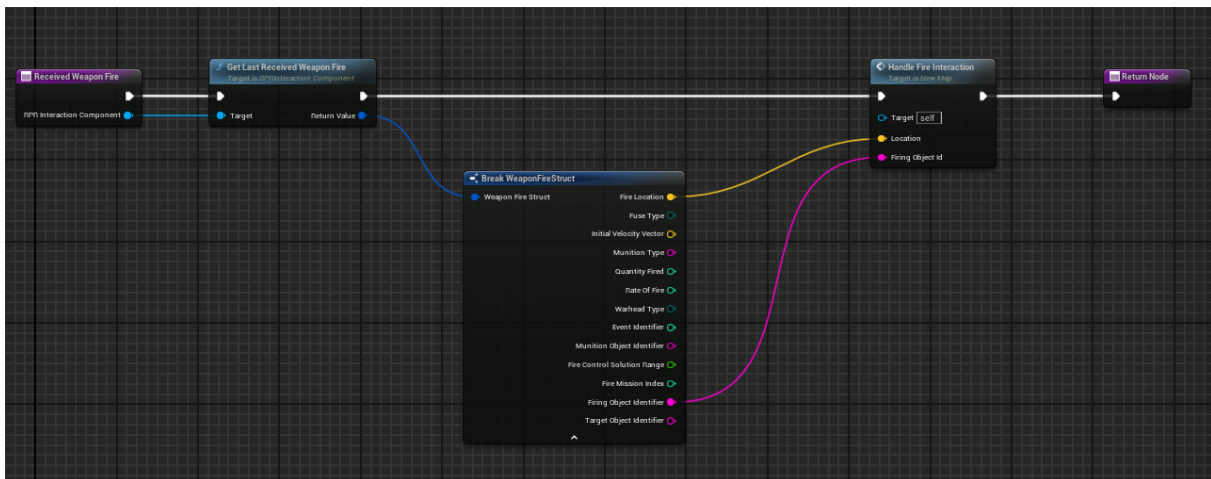


Figure 21. Retrieving interaction data.

Once the data has been retrieved, the interaction can be reflected in the Unreal Engine. In this example it could be a small explosion, provided by the model, when a weapon is fired.

The **RPR interaction Component** is a component of the **RPR Interaction Manager** Actor defined in **RPRConnector** as **RPR Interaction Spawner** property. The next figure shows how to get it.

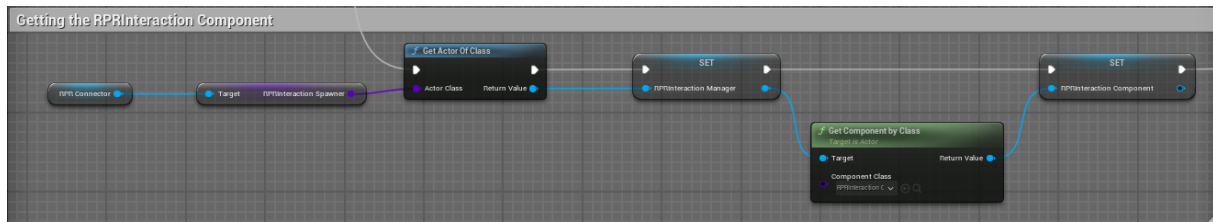


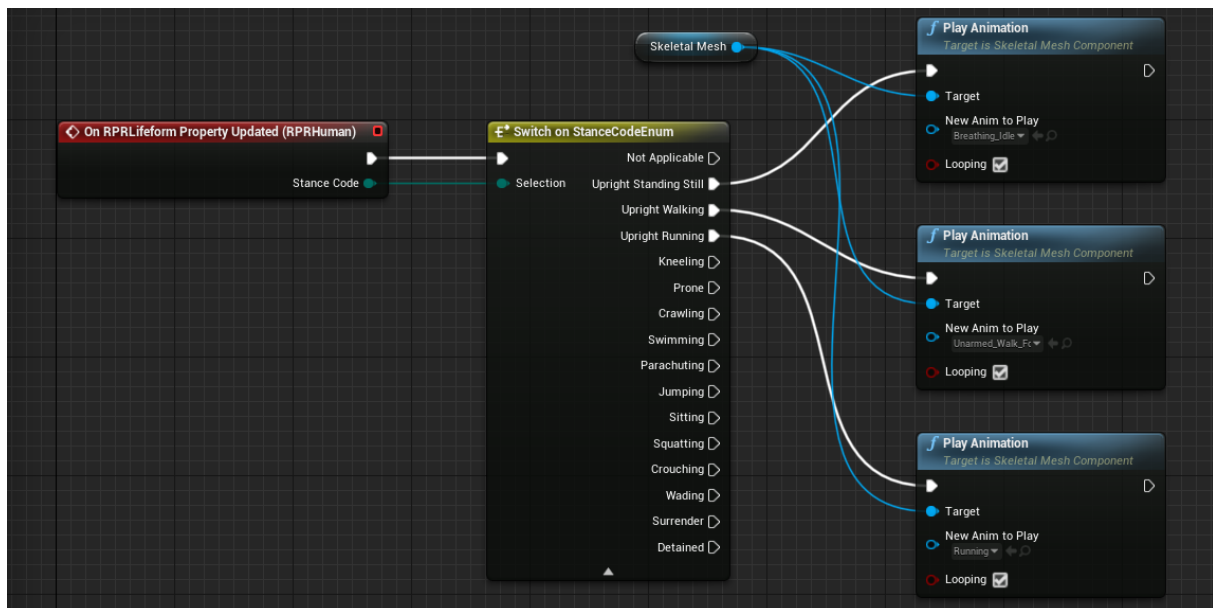
Figure 22. Getting RPR Interaction Component.

## 7.6. Human Actor Animations

Using the stance code attribute in the **RPRHumanComponent** Actor Component, different animations can be applied to blueprints with a skeletal mesh (E.g. a human). To do this, there needs to be a check done against the stance code. This can be done every time there is a property update for the actor, this is when the `OnRPRLifeformPropertyUpdated` event fires.

Here is an example of updating the animation using blueprints

*Animating a skeletal mesh.*



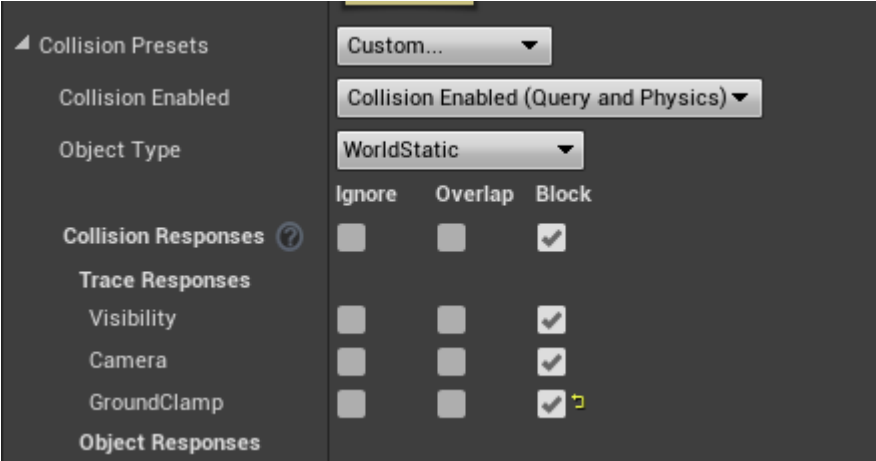
When there is a property update, it checks if the stance code is equal to 'Upright Walking', if it is then it sets a walking animation to play. The 'Looping' checkbox needs to be ticked as well, otherwise the animation will only play once then stop. More checks can be added to check for all possible values of stance code and set animations if they match.

The same concept can be applied to any property that would affect the appearance of an actor, e.g. lights. Some properties may need to be check each tick instead of using an event.

## 8. Ground Clamping

If the terrain is a static mesh or a landscape, in the collision settings of the static mesh, set the collision preset to **Custom** and tick block on the **GroundClamp** trace response.

*Set GroundClamp collision setting.*





## 9. Publishing

**NOTE** | Publishing is only available in the Standard edition of the plugin.

### 9.1. Publishing Local HLA Entities

Through the *RPR* component attached to an actor it is possible to publish a local Unreal Engine actor to the HLA federation. To do this, the *Is Local* boolean needs to be set to true in the *RPR* component of the actor instance.

A few other properties need to be set in the *RPR* component as well. When *Is Local* is enabled, the Dead Reckoning algorithm to be used can be chosen. The *Marking* needs to be set, as well as the *Entity Type*. Other properties, like *Force Identifier* and *Damage State* can also be set but are not required for the actor to be published. If not changed their default values will be used.

These properties can be set either in the editor, as shown below, or via blueprints by setting the property values using *Set* nodes.

Entity Appearance Attributes

Entity Attributes

Marking: GV-1

Force Identifier: Friendly

Damage State: No Damage

Entity ID: -1

Entity Identifier:

Entity Type: 1.1.0.0.0.0.0

Dead Reckoning Algorithm: 2 - FPW, Position is extrapolated frc

IsPartOf

Articulated Parameters Attributes

Articulated Parts Array: 0 Array elements

Settings

Location Interpolation Time: 5.0

Use Internal Location Rotation: ☒

Use Ground Clamping: ☒

HLA

Is Local: ☒

Instance Name:

Figure 23. Setting properties to the actor to publish.

While the project is running, any changes to any of these properties will be reflected in the local HLA entity that is published.

Spatial data about the actor can also be published. To do this there are some helper functions for setting the spatial data.

| Function                   | Description                                                                                                                                                                       |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Set Orientation to publish | The current orientation (in deg) of the actor. Can use <i>GetActorRotation</i> node.                                                                                              |
| Set Velocity to publish    | The current velocity (in m/s) of the actor. If there is a movement component the <i>GetVelocity</i> node might be able to be used, otherwise velocity will have to be calculated. |

| Function                        | Description                                                                                       |
|---------------------------------|---------------------------------------------------------------------------------------------------|
| Set Location to publish         | The current location of the actor in Unreal Engine coordinates. Can use <i>GetActorLocation</i> . |
| Set Acceleration to publish     | The current acceleration (in m/s²) of the actor. Will have to be calculated.                      |
| Set Angular Velocity to publish | The current angular velocity (in rads/s) of the actor. Will have to be calculated.                |

An example of the process using blueprints is shown below.

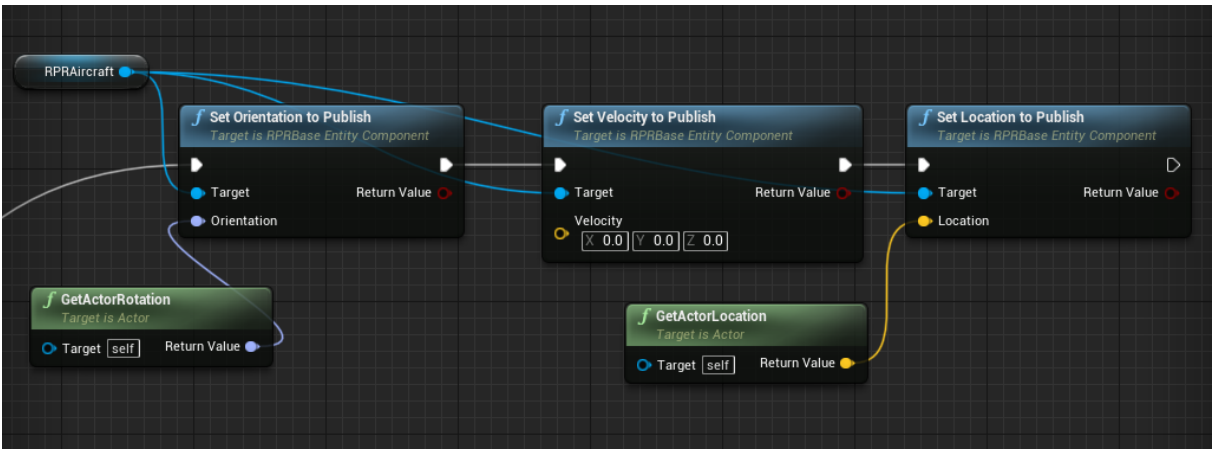


Figure 24. Updating properties via blueprint.

The plugin checks if there is any new data to send each tick. If there is no change in the data, or an updated spatial is not outside the set dead reckoning threshold, then no update will be sent.

A custom entity identifier can be chosen for an actor instance to be published by changing the Entity ID field. If the chosen entity identifier is already in use, the closet valid entity identifier will be chosen to replace it. Leaving the entity identifier field as -1 will automatically assign a free entity identifier to the published actor.



Figure 25. Setting the EntityId.

The Instance name is set automatically by the RTI if the field is left blank, otherwise the plugin will try to create the instance with the chosen instance name. An error is returned if the instance name already exists and the instance is not created.

## 9.2. Publishing Articulated Parts

Updating Articulated Parts data from a local Actor requires to add the Articulated Part

element to an array. This can be done using **Add Articulated Parts To Array** node or **Add Articulated Part Stuct To Array** node, then updating the value of each element.

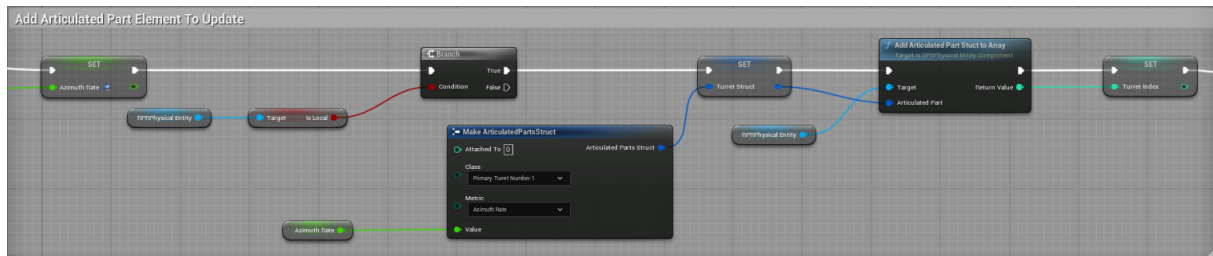


Figure 26. Adding an articulated part element to update.

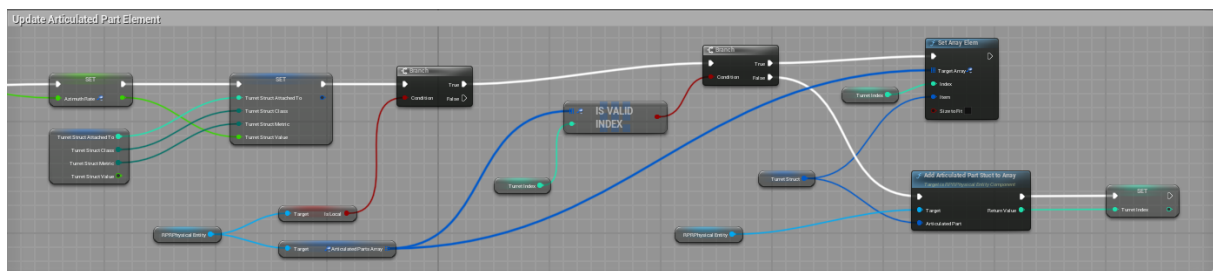


Figure 27. Updating the articulated part element.

Once it is no more necessary to update an Articulated Part element or all Articulated Part elements, use **Remove Articulated Part from Array** or **Empty Articulated Part Array** nodes.

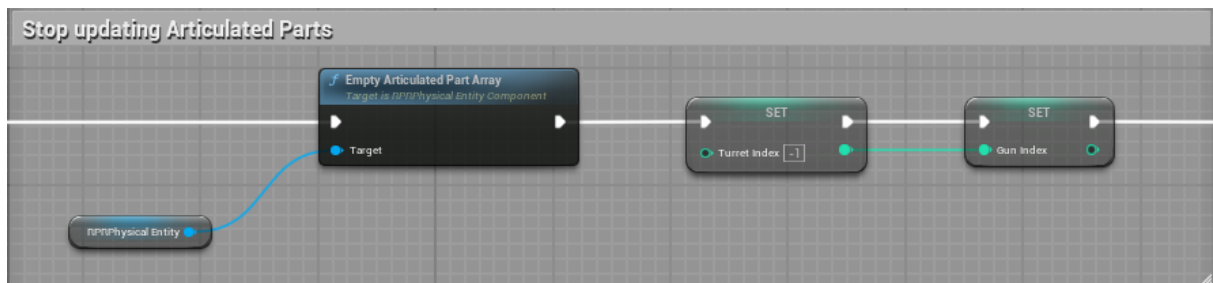


Figure 28. Stop updating Articulated Parts.

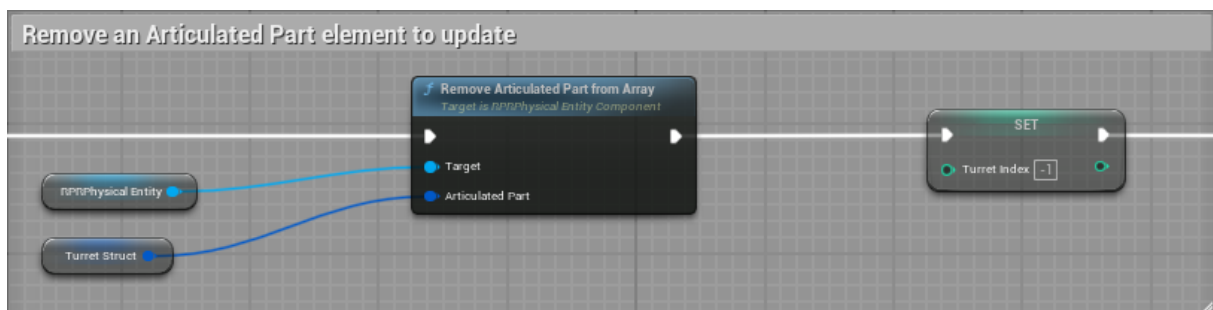


Figure 29. Remove an Articulated Part Element To Update.

## 9.3. Publishing Local HLA Interactions

To publish interactions, get the interaction handler actor from the level. This actor will be

based on **Base Interaction Manager** class. Using the RPR Interaction Component in the interaction handler, call the **Add X Interaction To Publish** node to add the interaction to an array of interactions to be sent. Adding the interaction to the array will not send it straight away. Call **Publish Interaction** to send all the interactions stored in the array.

An example of sending a Weapon Fire interaction:

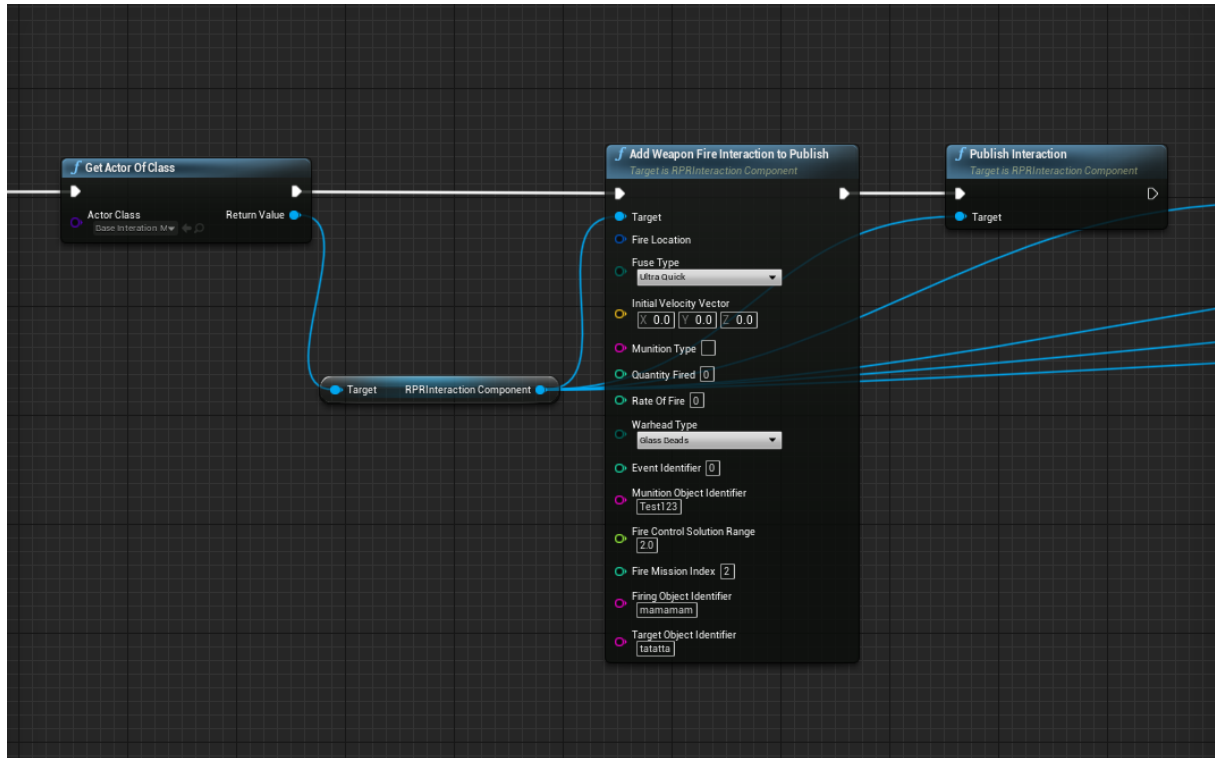


Figure 30. Publishing interactions.

Interactions can be send from any blueprint that has access to the actors in the current level.

## 10. Running the project

The project is now ready to play.

Before playing the game, make sure that a RTI execution is running and the project is calling the chosen Join Federation function somewhere.

## 11. The RPRConnector Sample project

The plugin is provided with a sample project named `PitchUEConnector.uproject`. It is not provided with the free version of the plugin.

This sample project provides:

- A level called `NewMap` that includes a **RPRConnector** actor, with a default configuration, and a **Geo Referencing System** actor, configured with a reference location close to Porquerolles Island, France. Note that the project does not provide any terrain.
- A Level blueprint prepared to join the federation.
- A Mapping using generic Actors (geometrical shapes). These actors are ready to handle data received from the federation.
- User inputs.

This sample proposes some features like:

- A Menu UI to connect/disconnect to/from the federation, edit the connection settings
- A Mapping UI to edit the mapping during the runtime
- An Entity Identification Widget, which display marking and symbol associated to the entities
- An Entity Information Widget, which display some information about a selected entity
- Some features to position the point of view like attaching to an entity with zoom and rotation

### 11.1. Building the project

To build the project, follow these steps:

- a. Right click `PitchUEConnector.uproject` and select *Generated Visual Studio project files*
- b. Double click `PitchUEConnector.sln` to open the Visual Studio solution
- c. In Visual Studio, go *Build → Build Solution*. This will build the Unreal Engine project and the Unreal Engine Connector Plugin

Alternatively, it is possible to build the project with following steps:

- a. Double-click on `PitchUEConnector.uproject`
- b. Answer 'Yes' to the Missing `PitchUEConnector` Modules message box.

Note that this alternative does not generate the Visual Studio solution.

## 11.2. Displaying the Menu UI

To display the Menu UI, in the Unreal Engine Editor, open the Level Blueprint then add this blueprint functions, for example to the Event BeginPlay:

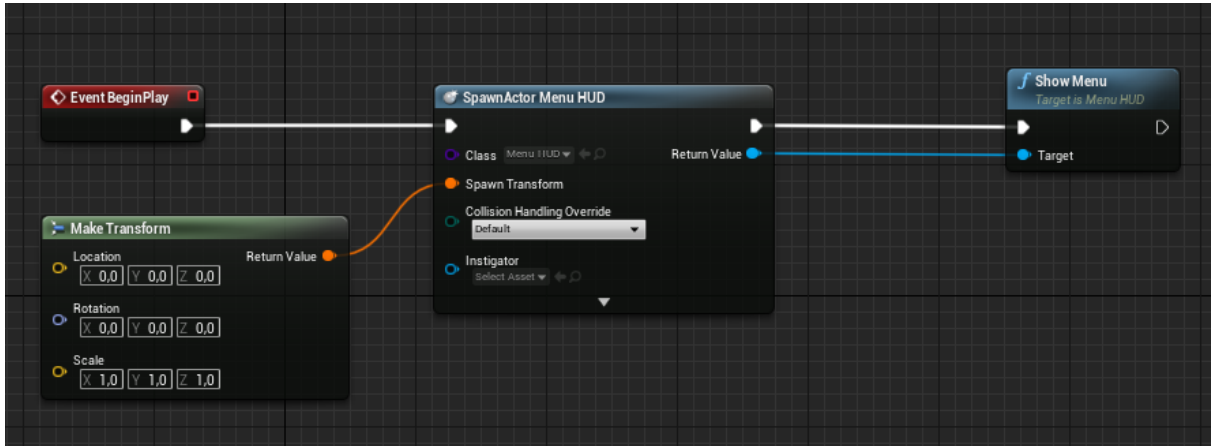


Figure 31. Displaying the Menu UI in the level blueprint.

Then compile and save.

The Menu UI looks like:

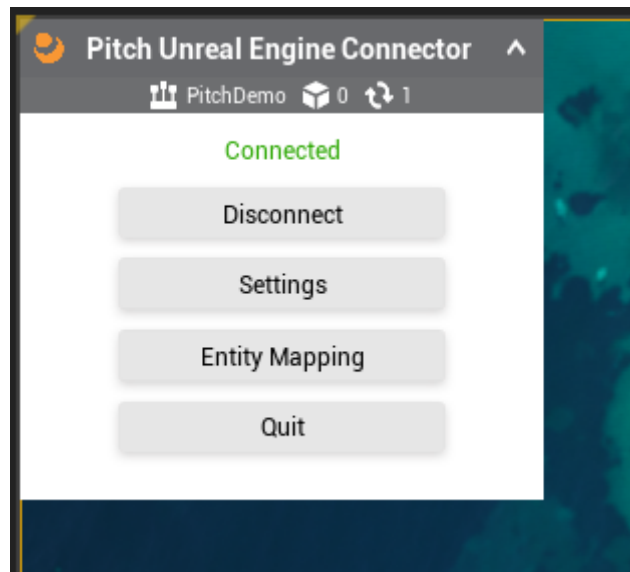


Figure 32. The Menu UI

It displays:

- At the top:
  - The name of the federation
  - The number of objects
  - The number of received messages, including discoveries, updates, interactions



- Indicates the plugin is connected or not to the federation
- A button to connect or disconnect
- A button to open the Settings UI
- A button to open the Entity Mapping UI
- A button to quit

## 11.3. User inputs

The sample project comes with a set of pre-configured user inputs which allows to do some actions like displaying an Entity Identification widget or attach the point of view on a specific entity. To be able to use these inputs, the following code shall be added to the level blueprint:

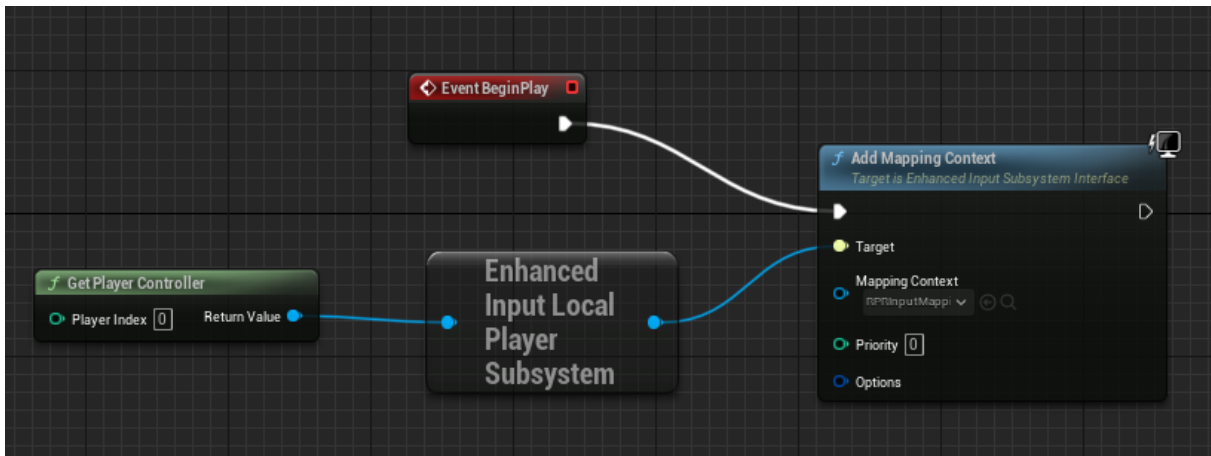


Figure 33. Enabling inputs mapping

## 11.4. Displaying the Entity Identification Widget

Enabling/disabling display of the Entity Identification Widget use the *ShowEntityWidget* input action and shall be bound to the *ToggleShowEntityIdWidgets* function as shown below:

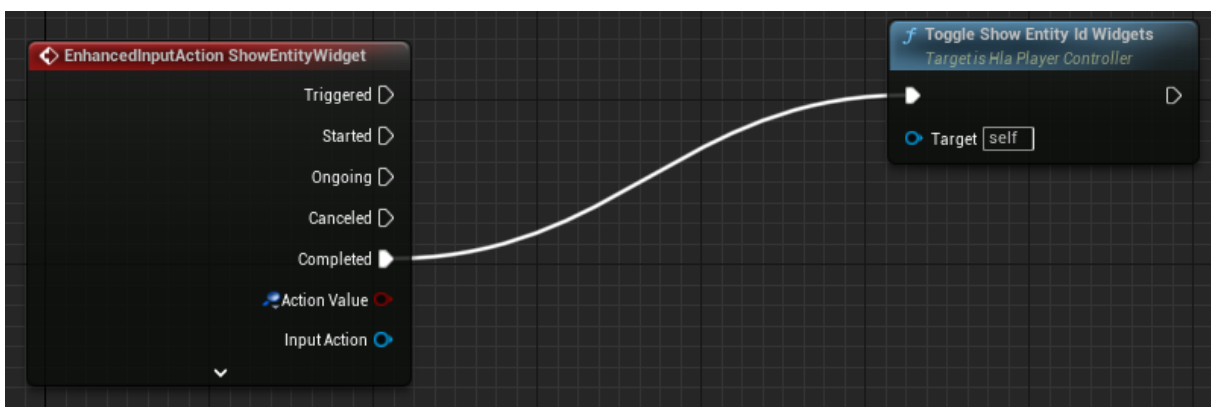


Figure 34. Binding ShowEntityWidget action (in BP\_RPRPlayerController asset)

This action is mapped in *RPRInputMappingContext* asset with the key x.

## 11.5. Displaying the Entity Information Widget

In game, click on an entity, or if displaying entity identification widgets is enabled its marking or symbol to select it.

When an entity is selected, additional entity information UI is displayed below the entity. The UI contains the instance name, entity ID, marking, entity type, position, velocity, force identifier and damage state. The UI will stay on a screen until the entity is unselected or until the X button located in the upper right corner is clicked.

## 11.6. Point of view

In game, click on an entity, or if displaying entity identification widgets is enabled its marking or symbol to select it.

The input action *AttachToEntity* shall be bound to the function *ToggleAttachToEntity*.

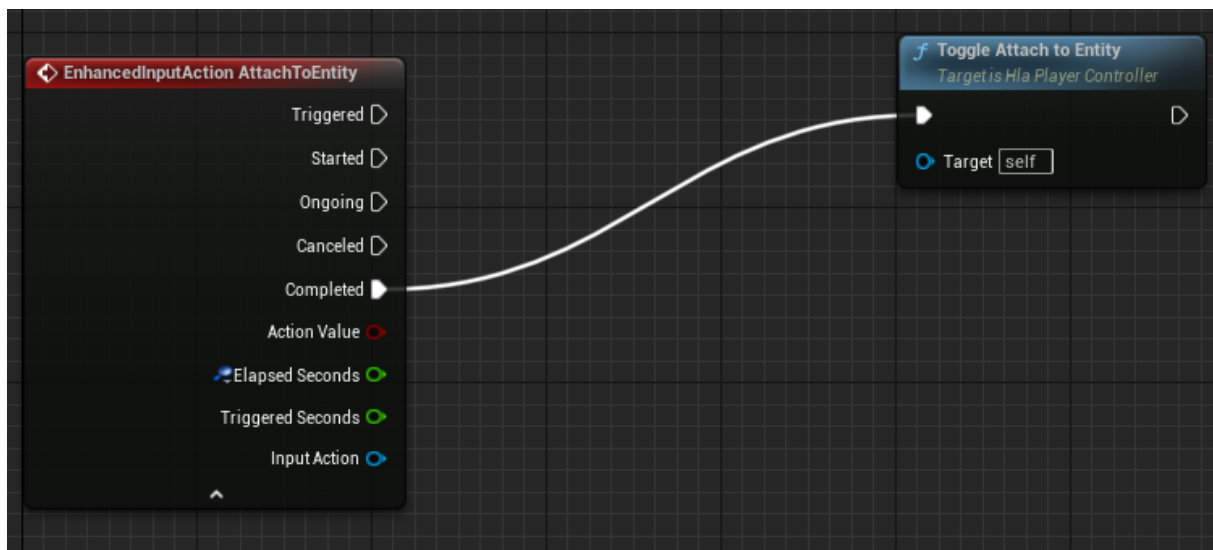


Figure 35. Binding *AttachToEntity* action (in *BP\_RPRPlayerController* asset)

Clicking the e key (or the key mapped to the action in the mapping context) will attach to the selected camera and follow the entity.

When attached to an entity, press and hold the left or right mouse button and move the mouse to rotate around the entity. Use the scroll wheel to zoom in and out to and from the entity.

These actions are bound in the sample project in *BP\_RPRCharacter* asset.

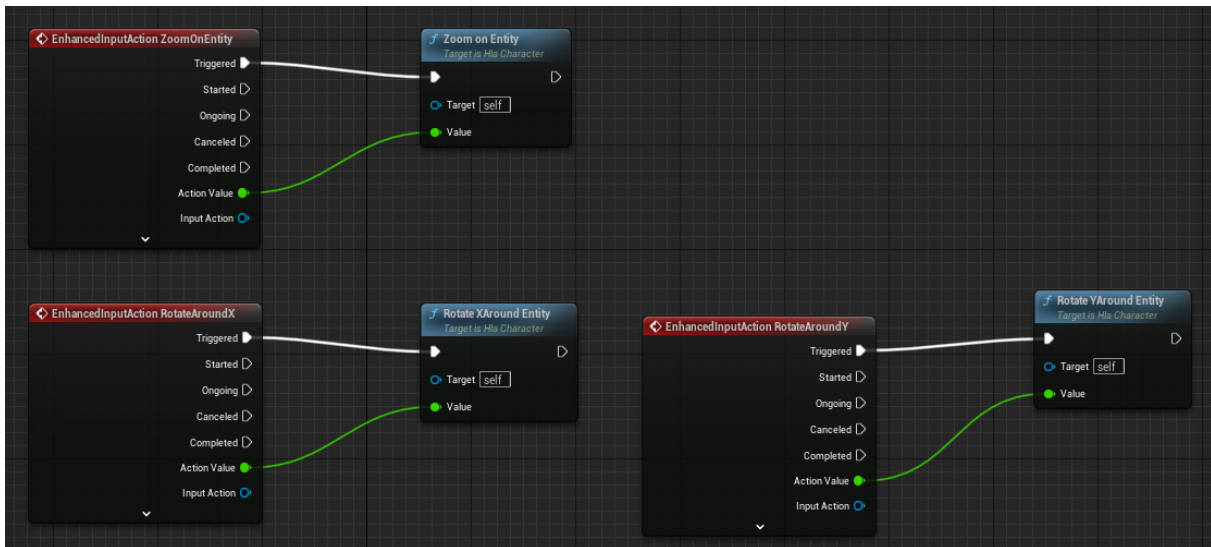


Figure 36. Binding point of view actions

Clicking the e key (or the key mapped to the action in the mapping context) will detach from an entity, if attached.

## 11.7. Working with previous versions of Unreal Engine

The sample project `PitchUEConnector.uproject` has been developed using Unreal Engine 5.5. To use a previous version of Unreal Engine, follow these steps:

1. Right-click on `PitchUEConnector.uproject` file then `Switch Unreal Engine version...`; Select the target UE version in the message box.
2. Open the generated Visual Studio solution file and (re)build it.
3. Open the UE project. A error message pops up indicating that the UE map included in the project is from a newer version. Click OK.
4. Create a new level as described in [Section 5](#) section.

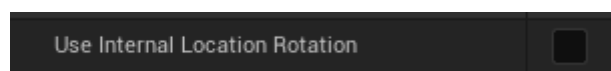
## 12. Troubleshooting & known issues

This section defines some common errors and how to resolve them.

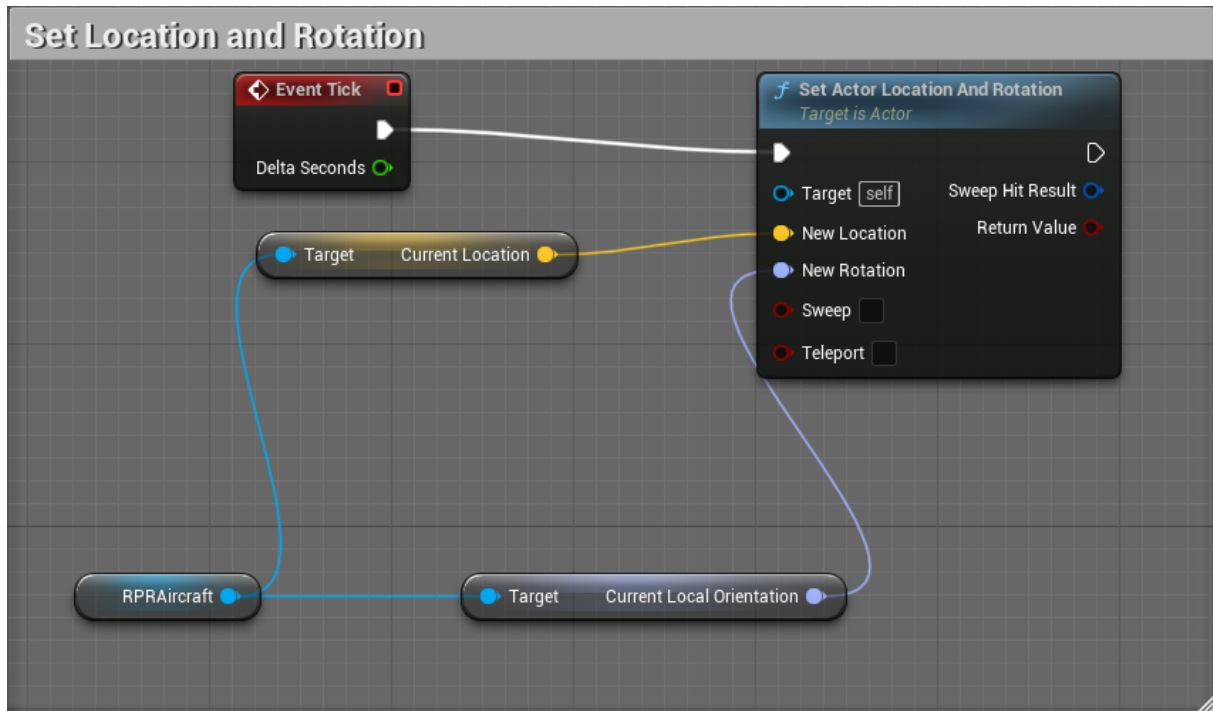
| Issue                                                                                                                                 | Solution                                                                                                                                                                                                                                                                                                                                  |
|---------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Failed to load pRTIEvolvedDriver64.dll. HLA will not be functional!                                                                   | Usually caused by one of two things -<br>1. Path to the DLL is too long<br>2. The DLL does not find RTI libraries, probably because it is missing from PATH and/or CLASSPATH environment variables. When using pRTI, check prti_federate_quick_start guide. For other RTIs, check their documentation.                                    |
| HlaConnectException: Unable to get RTI ambassador from RtiDriver                                                                      | This error usually happens when RTI libraries cannot be found, see above.                                                                                                                                                                                                                                                                 |
| Failed to load DisDriver64.dll. DIS will not be functional!                                                                           | Path to the DLL is too long                                                                                                                                                                                                                                                                                                               |
| Actors jittering in the display                                                                                                       | It could happen that some actors, mainly aircrafts, whose position is updated in the C++ code looks jittering in the display. To workaround this behaviour, follow the steps described in <a href="#">Section 12.1</a> .                                                                                                                  |
| When using the Origin at Planet Center setting in the <b>Geo Referencing System</b> plugin, the location and rotation may be invalid. | Recommendations are:<br>- Prefer using a projected CRS or<br>- Disable Use Internal Location Rotation and Use Ground Clamping properties in the <b>RPRComponent</b> of the blueprint model and compute actor location and rotation manually from Current RPR Location and Current RPR Orientation properties of the <b>RPRComponent</b> . |

### 12.1. Set Actor Location and Rotation in blueprint model

1. Open the 3D model blueprint used by the actor. In its *RPRComponent*, uncheck the *Use Internal Location Rotation* property.



2. Add this code in the model blueprint



3. Check the property *Start with Tick Enabled* for the model





Pitch Technologies

Repslagaregatan 25, S-582 22 Linköping, Sweden

Tel: +46 (0)13 470 55 00

Web: <https://www.pitchtechnologies.com>, Email: [info@pitchtechnologies.com](mailto:info@pitchtechnologies.com)

[1] Albeit *StartResume* and *StopFreeze* interactions have less than nine parameters, the same technique is applied